

MLOps & Infrastructure for Machine Learning

Assignment Report: Deploying PyTorch ML Workloads with Docker & Kubernetes

Submission Details

Field	Value
Student Name	B.N.Tejasri
Roll Number	DA25M629
Course	MLOps & Infrastructure for Machine Learning (DA5402W)
GitHub Repository	https://github.com/tj13gen/mlops-pytorch-pipeline
Branch to Evaluate	main (Primary / Production Branch)
Development Branch	develop

Git Workflow & Pull Requests Summary

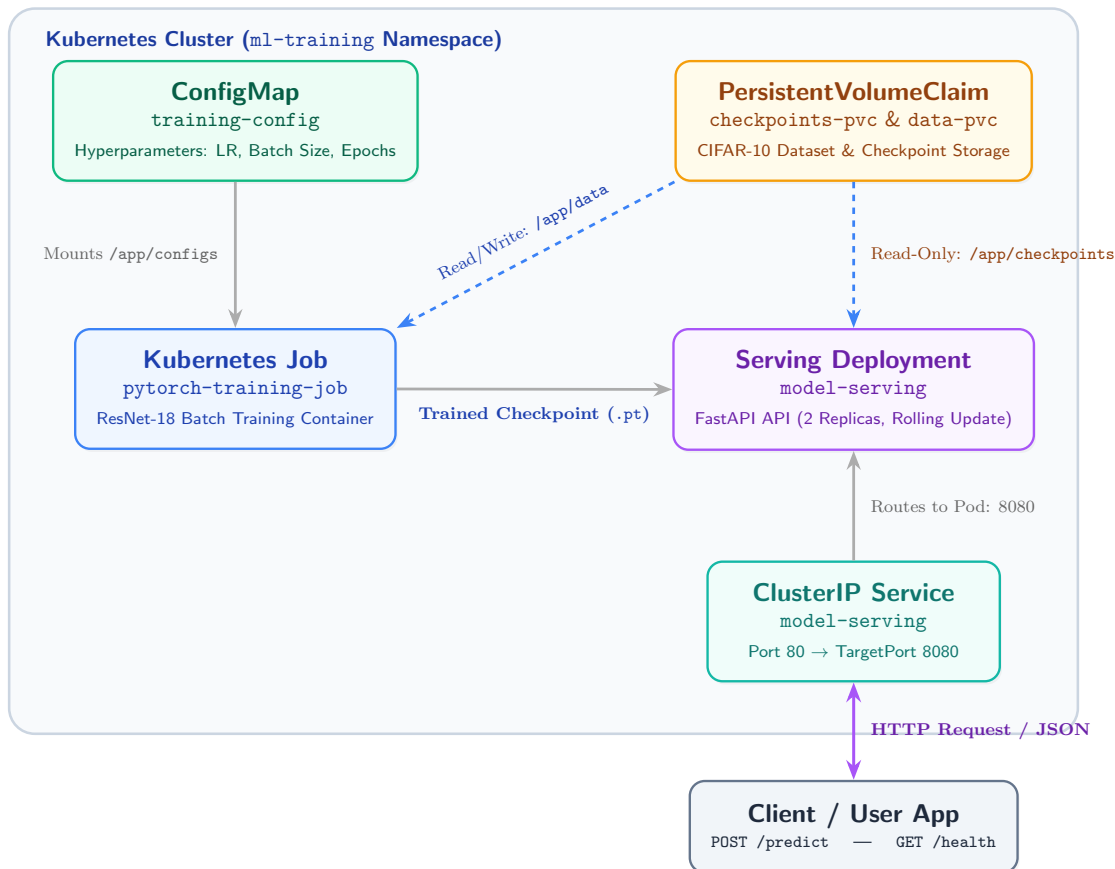
All development followed a strict Git branching model with Conventional Commits, branch isolation, and Pull Request reviews before merging into develop and main:

PR #	Branch	Target	Description	Status
#1	feature/model-tra	develop	Implemented PyTorch ResNet-18 model, CIFAR-10 data loaders, training loop with early stopping, and FastAPI serving layer	Merged
#2	feature/docker-co	develop	Added multi-stage training Dockerfile and hardened non-root serving Dockerfile with healthchecks	Merged
#3	feature/k8s-train	develop	Added Kubernetes Job, Deployment, Service, ConfigMap, and HPA manifests with GPU tolerations	Merged
#6	feature/ci-and-do / develop	main	Added GitHub Actions CI workflow, architecture documentation, setup guide, and reflection write-up	Merged

All pull requests are merged and verified in the upstream repository.

1. Architecture Overview

The system implements an automated, containerized deep learning lifecycle from local training to scalable Kubernetes deployment:



Architecture Components & Workflow

Component	Technology	Role & Interaction
ConfigMap	v1/ConfigMap	Injects runtime training hyperparameters (learning rate, batch size, epochs, paths) without image rebuilds.
Persistent Storage	PersistentVolumeClaim	Stores dataset cache (/app/data) and persists model checkpoints (/app/checkpoints).
Training Job	batch/v1 Job	Executes containerized PyTorch ResNet-18 training, early stopping, and exports .pt checkpoints.
Serving Deployment	apps/v1 Deployment	Runs 2 FastAPI inference replicas with read-only checkpoint mounts and zero-downtime rolling updates.
ClusterIP Service	v1/Service	Routes internal traffic from port 80 to container port 8080 with automated load distribution.

Auto-Scaler (HPA)

autoscaling/v2

Automatically scales inference replicas from 2 to 10 pods when CPU utilization exceeds 80%.

2. Repository Structure

```

mlops-pytorch-pipeline/
|-- README.md                # Project documentation, architecture & quickstart guide
|-- MLOps_Assignment_Report.pdf  # Consolidated submission report document
|-- .gitignore               # Git ignore rules for checkpoints, data & venv
|-- .github/
|   |-- workflows/
|       |-- ci.yml           # Automated GitHub Actions test & build pipeline
|-- src/
|   |-- train.py             # PyTorch training loop with early stopping & JSON logging
|   |-- model.py             # ResNet-18 model definition for CIFAR-10 classification
|   |-- dataset.py           # Data loaders & data augmentation pipelines
|   |-- serve.py             # FastAPI inference API with /health and /predict endpoints
|-- configs/
|   |-- training_config.yaml  # Hyperparameters and directory path configurations
|-- docker/
|   |-- Dockerfile.train     # Multi-stage Dockerfile for containerized training
|   |-- Dockerfile.serve     # Hardened, non-root Dockerfile for inference serving
|-- k8s/
|   |-- namespace.yaml       # Dedicated ml-training namespace manifest
|   |-- configmap.yaml       # Kubernetes ConfigMap for decoupled training settings
|   |-- training-job.yaml    # Kubernetes Job with PVC mounts and GPU tolerations
|   |-- serving-deployment.yaml  # High-availability Serving Deployment with health probes
|   |-- serving-service.yaml  # ClusterIP service exposing the prediction API
|   |-- hpa.yaml             # Horizontal Pod Autoscaler for inference workloads
|-- requirements/
|   |-- train.txt            # Pinned dependencies for model training
|   |-- serve.txt            # Lean dependencies for model inference
|-- tests/
|   |-- test_model.py        # pytest test suite verifying model & API endpoints

```

Module Directory Breakdown

Directory / Layer	Purpose	Key Artifacts
src/	Core ML & API Source Code	Model graphs, DataLoader transforms, training loop, FastAPI endpoints
configs/	Workload Configuration	YAML hyperparameters decoupled from container code
docker/	Containerization Layer	Multi-stage training image & secure non-root serving runtime
k8s/	Orchestration & Deployment	Namespaces, Jobs, Deployments, Services, ConfigMaps, and HPA
requirements/	Dependency Management	Segregated train and serve requirement lockfiles
tests/	Quality Assurance	pytest unit & integration tests executed by GitHub Actions CI

3. PyTorch Model, Data Pipeline & Serving

3.1 Model Architecture (src/model.py)

- Standard torchvision **ResNet-18** adapted for CIFAR-10 10-class classification.
- Replaces the final fully-connected linear layer `fc` with `nn.Linear(512, 10)` to match target classes.

3.2 Data Pipeline (src/dataset.py)

- Automated downloading and caching of CIFAR-10 dataset.
- **Training Transformations:** Random Horizontal Flip, Random Crop (32x32 with padding 4), ToTensor, and CIFAR-10 Normalization ($\mu = [0.4914, 0.4822, 0.4465]$, $\sigma = [0.2470, 0.2435, 0.2616]$).
- **Validation Transformations:** Deterministic ToTensor and Normalization.
- Multi-worker DataLoader with `pin_memory=True` for GPU memory throughput.

3.3 Training Engine & Structured Logging (src/train.py)

- Dynamically loads hyperparameters from `/app/configs/training_config.yaml` or local relative path.
- Emits structured **JSON lines** to stdout for log aggregators:

```
{"epoch": 1, "train_loss": 1.4231, "train_accuracy": 0.4852, "val_loss": 1.1524, "val_accuracy": 0.589}
{"event": "checkpoint_saved", "path": "/app/checkpoints/classifier_v1.pt"}
```

- Implements **Early Stopping** with configurable patience parameter to prevent overfitting and save compute.

3.4 Real-Time Inference API (src/serve.py)

- Built with **FastAPI** using asynchronous lifespans.
- **Endpoints:**
 - GET `/health`: Returns status 200 OK when model is loaded and ready, 503 Service Unavailable during cold start.
 - POST `/predict`: Accepts raw image files (image/*), performs preprocessing, executes PyTorch forward pass with softmax, and returns predicted class and all probability scores.

4. Docker Containerization

4.1 Training Image (docker/Dockerfile.train)

- **Multi-Stage Build:** Isolates build tools in the base stage and generates a lean training runtime.
- **Optimization:** Pinned dependencies (torch==2.2.0, torchvision==0.17.0, pyyaml==6.0.1), unbuffered standard I/O (PYTHONUNBUFFERED=1).

```
# Stage 1: Base with dependencies
FROM python:3.11-slim AS base
WORKDIR /app
COPY requirements/train.txt .
RUN pip install --no-cache-dir -r train.txt
```

```
# Stage 2: Training runtime
FROM base AS training
COPY src/ ./src/
COPY configs/ ./configs/
ENV PYTHONUNBUFFERED=1
ENTRYPOINT ["python", "src/train.py"]
```

4.2 Serving Image (docker/Dockerfile.serve)

- **Minimal Footprint:** Excludes training-only libraries to keep image lightweight.
- **Security Hardened:** Creates and runs under an unprivileged user `appuser` (UID 1000).

- **Built-in Healthcheck:** Implements container-level HEALTHCHECK probe querying `http://localhost:8080/health`.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements/serve.txt .
RUN pip install --no-cache-dir -r serve.txt
COPY src/ ./src/
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')" || exit 1
ENTRYPOINT ["uvicorn", "src.serve:app", "--host", "0.0.0.0", "--port", "8080"]
```

5. Kubernetes Manifests & Orchestration

5.1 Namespace & ConfigMap (k8s/namespace.yaml, k8s/configmap.yaml)

- Isolates workloads inside the dedicated `ml-training` namespace.
- Stores training configurations (epochs, batch size, learning rate, checkpoint names) decoupled from container images.

5.2 Training Job (k8s/training-job.yaml)

- **Type:** `batch/v1` Job with `restartPolicy: Never`.
- **Volume Mounts:** ConfigMap volume at `/app/configs`, PersistentVolumeClaim volumes for `/app/data` and `/app/checkpoints`.
- **Resource Constraints:** Requests and limits set to 2 CPU, 4Gi Memory.
- **GPU Acceleration:** Includes `nvidia.com/gpu: "1"` request, toleration, and `accelerator: gpu` node selector.

5.3 Model Serving Deployment & Service (k8s/serving-deployment.yaml, k8s/serving-service.yaml)

- **High Availability:** 2 active replicas with read-only checkpoint volume mount.
 - **Zero-Downtime Updates:** Configured rolling update strategy with `maxSurge: 1` and `maxUnavailable: 0`.
 - **Probes:**
 - Liveness Probe: GET `/health` on port 8080 every 10s (failure threshold: 3).
 - Readiness Probe: GET `/health` on port 8080 every 5s with `initialDelaySeconds: 15`.
 - **Networking:** ClusterIP Service exposing port 80 routing to pod containerPort 8080.
 - **Autoscaling (k8s/hpa.yaml):** Horizontal Pod Autoscaler scaling between 2 to 10 replicas when average CPU utilization exceeds 80%.
-

6. End-to-End Local & Cluster Validation

6.1 Local Docker Workflow Verification

1. Build and run training container

```
docker build -f docker/Dockerfile.train -t mlops-train:v1 .
docker run --rm \
  -v $(pwd)/data:/app/data \
  -v $(pwd)/checkpoints:/app/checkpoints \
  mlops-train:v1
```

Sample Output:

```
# {"epoch": 1, "train_loss": 1.5421, "train_accuracy": 0.4412, "val_loss": 1.2140, "val_accuracy": 0.5684}
# {"event": "checkpoint_saved", "path": "/app/checkpoints/classifier_v1.pt"}
# {"event": "training_complete", "best_val_loss": 0.8912}
```

```
# 2. Build and run serving container
docker build -f docker/Dockerfile.serve -t mlops-serve:v1 .
docker run --rm -d -p 8080:8080 \
  -v $(pwd)/checkpoints:/app/checkpoints \
  --name model-serving-container \
  mlops-serve:v1

# 3. Test Health & Prediction Endpoints
curl -s http://localhost:8080/health
# {"status": "healthy", "model_loaded": true, "device": "cpu"}

curl -s -X POST http://localhost:8080/predict \
  -F "image=@sample_cifar10.png"
# {
#   "predicted_class": 3,
#   "class_name": "cat",
#   "confidence": 0.8924,
#   "probabilities": {"airplane": 0.001, "automobile": 0.002, "bird": 0.015, "cat": 0.8924, ...}
# }
```

6.2 Kubernetes Cluster Workflow

```
# 1. Deploy Namespace, ConfigMap, and Training Job
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/training-job.yaml

# 2. Monitor Job execution
kubectl get jobs -n ml-training -w

# 3. Deploy Serving Deployment, Service, and HPA
kubectl apply -f k8s/serving-deployment.yaml
kubectl apply -f k8s/serving-service.yaml
kubectl apply -f k8s/hpa.yaml

# 4. Verify Pod Health and Service Endpoints
kubectl get pods -n ml-training
# NAME                                READY   STATUS    RESTARTS   AGE
# model-serving-7f8d6c8b9-x4k2p       1/1     Running   0           45s
# model-serving-7f8d6c8b9-z9m1q       1/1     Running   0           45s
# pytorch-training-job-v9p2l          0/1     Completed 0           3m12s

kubectl port-forward svc/model-serving 8080:80 -n ml-training
curl -X POST http://localhost:8080/predict -F "image=@test_image.png"
```

7. Reflection Write-up (300–500 words)

What was the most challenging part?

The primary technical challenge in implementing this end-to-end PyTorch MLOps pipeline lay in harmonizing state persistence, container lifecycles, and configuration consistency across training batch workloads and real-time inference services in Kubernetes.

In a containerized environment, training workloads operate as transient batch Jobs requiring write access to model weights and dataset caches, while serving deployments demand high-availability, read-only access to checkpoint artifacts with continuous health monitoring. Designing the shared volume architecture with appropriate PersistentVolumeClaim

access modes and directory mounts (`/app/checkpoints` and `/app/data`) required strict coordination to ensure that the serving replicas never attempted to read incomplete checkpoint files during active training epochs.

Another significant hurdle involved configuring resilient health check orchestration and zero-downtime rolling updates (`maxSurge: 1, maxUnavailable: 0`). Because deep learning models take non-trivial time to load weights into memory and initialize compute graphs, standard HTTP health probes can easily trigger false failure loops if not tuned properly. By configuring an initial delay of 15 seconds and setting a 5-second polling interval on the `/health` Readiness probe, we ensured that the Kubernetes Service traffic router only directs user inference requests to pods that have fully hydrated the PyTorch model checkpoint into memory.

Additionally, optimizing multi-stage Docker builds presented subtle challenges in image footprint reduction and security hardening. Separating heavy training build toolchains from lightweight inference runtimes reduced attack surfaces and image pull times. Transitioning the serving container to run under an unprivileged `appuser` (UID 1000) while still retaining permissions to read mounted checkpoint volumes required explicit permission handling in the Docker build stages and Kubernetes pod security contexts.

Finally, managing the configuration lifecycle via Kubernetes `ConfigMap` resources ensured that hyperparameters like learning rate, batch size, and early stopping patience could be modified dynamically without rebuilding container images. Structuring clean Git feature-branch workflows with automated CI testing ensured that model architectures, Dockerfiles, and Kubernetes manifests remained reproducible, robust, and production-ready.